



PYTHON + LLM API + PYDANTIC | INTERMEDIATE DEVS

Build Your Own **LLM-as-Judge** Evaluation Harness

Skip the black box. Understand exactly how deepeval and RAGAS work by building a mini version yourself.



Swipe to build from scratch →



AI Coach John



FOLLOW



Repost

Why Build This Yourself

You will use libraries like RAGAS and deepeval eventually: build this first.

- ✓ **Black boxes until understood:** Libraries like RAGAS and deepeval are excellent, but they are black boxes until you understand what is happening inside them.
- ✓ **Uncover core mechanisms:** Building your own reveals exactly where bias creeps in, why prompt design matters so much, and what "calibration" actually means.
- ✓ **Effortless customization:** Once you have built this, reading (and customizing) any evaluation library's source code becomes trivial because you already know the shape.
- ✓ **Production pattern:** This is also a legitimate production pattern: many teams outgrow generic libraries and need custom metrics that only a hand-rolled harness can express.

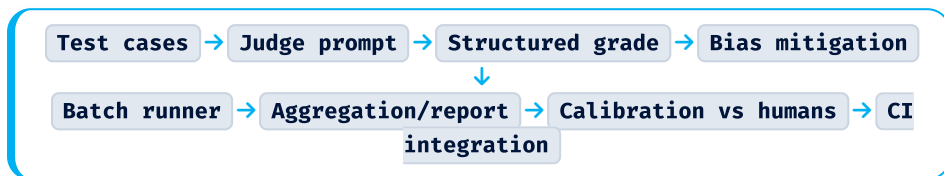
✂ What We Are Building

A framework that can score any LLM output against a rubric using another LLM as the grader, featuring proper bias mitigation, structured output, batch running, and reporting.



Architecture Roadmap

What a real evaluation harness is made of: Here is the exact pipeline flow and the 8 core components we will build in order.



The 8 Components We Will Build

01 Test case structure
Data schema

02 Judge prompts
Pointwise & pairwise

03 Output parsing
Structured Pydantic

04 Bias mitigation
Position & verbosity

05 Metric abstraction
Reusable base class

06 Batch runner
Dataset execution

07 Aggregation & report
Stats & human calibration

08 CI/CD integration
Regression testing

terminal setup

```
uv add pydantic openai anthropic pandas scipy
```



AI Coach John



FOLLOW

 Repost

Building Block 1: The Test Case

Defining what a single evaluation unit looks like: Every evaluation needs a consistent structure describing what was asked, what context (if any) was given, and what the model produced. This is the atomic unit everything else operates on.

test_case.py

```
from pydantic import BaseModel
from typing import Optional

class TestCase(BaseModel):
    input: str # The question/prompt
    given_to_the_model
    actual_output: str # What the model under
    test produced
    expected_output: Optional[str] = None # Reference answer if
    you have one
    context: Optional[list[str]] = None # Retrieved context for
    RAG evaluation
    metadata: Optional[dict] = None # Anything else: model
    name, run id, etc.
```

Why Pydantic Matters Here

Using **pydantic** here is not optional decoration: it enforces that every test case has a consistent shape before it hits your judge logic, which prevents a huge class of "why did this metric silently return garbage" bugs later.



Pointwise vs Pairwise Grading

The two fundamental ways an LLM can judge something: Choosing the right mode determines whether your metric is reliable or noisy.

★ Pointwise

The judge scores a single output on its own against a rubric (e.g., "rate this answer's faithfulness from 1 to 5").

✓ When to use:

Better for absolute, comparable metrics you want to track over time (such as faithfulness score trending across releases).

⚖️ Pairwise

The judge compares two outputs head-to-head and picks which is better (e.g., "which of these two answers is more helpful?").

✓ When to use:

Better for subjective quality comparisons (is version A's tone better than B's?). LLMs are much more reliable at relative judgments.



Key Rule: Relative head-to-head comparisons are consistently more stable than absolute scoring across different LLM judge families.



AI Coach John



FOLLOW

 Repost



Judge Prompt Design: Pointwise Metric

Writing a prompt that produces a reliable score: Defines the rubric explicitly, asks for reasoning before a score (not just a number), and constrains output format.

```
prompts/pointwise.py
```

```
FAITHFULNESS_PROMPT = """You are an expert evaluator. Score how
faithful the
ANSWER is to the CONTEXT on a scale of 1 to 5.
5 = every claim is fully supported by the context
3 = some claims are supported, some are not verifiable from
context
1 = the answer contradicts or ignores the context entirely

Context: {context}
Answer: {answer}

First, briefly reason step by step about which claims are
supported.
Then output your final score as JSON: {"reasoning": " ... ",
"score": <int>}}
"""
```



Why Chain-of-Thought Matters

Asking for reasoning before the score consistently produces more accurate and more consistent scores than asking for a bare number: the reasoning step forces the judge model to actually engage with the specifics instead of pattern-matching to a "looks fine" default.





Judge Prompt Design: **Pairwise Comparison**

Writing a prompt that compares two outputs fairly: Presents both outputs, asks for a reasoned comparison, and forces a structured verdict (critical for order randomization).

```
prompts/pairwise.py
```

```
PAIRWISE_PROMPT = """You are comparing two AI assistant responses  
to the same question.  
Determine which response is more helpful, accurate, and complete.
```

```
Question: {question}  
Response A: {response_a}  
Response B: {response_b}
```

```
Reason step by step about the strengths and weaknesses of each.  
Then output JSON: {"reasoning": "...", "winner": "A" | "B" |  
"tie"}}
```

Always Include a "Tie" Option

Forcing a winner when responses are genuinely equivalent introduces noise into your metric and makes two nearly-identical prompt versions look artificially different from each other.



Structured Output Parsing

Making sure the judge's output is actually usable: Uses Pydantic to define the exact shape of a valid response and validates raw output against it to catch malformed grading.

parser.py

```
import json
from pydantic import BaseModel, ValidationError

class PointwiseGrade(BaseModel):
    reasoning: str
    score: int

def parse_grade(raw_output: str) → PointwiseGrade:
    try:
        data = json.loads(raw_output)
        return PointwiseGrade(**data)
    except (json.JSONDecodeError, ValidationError) as e:
        raise ValueError(f"Judge returned invalid output: {raw_output}") from e
```

Fail Loudly on Bad Output

Never trust that the judge model will always return perfectly formatted JSON: always validate and fail loudly (not silently) on malformed output, since a silently-skipped bad grade can quietly skew your aggregate metrics without anyone noticing.



Bias Mitigation 1: Position Bias

LLM judges favor whichever answer they see first (or second): Research consistently shows systematic position bias. The fix is running the check twice with order swapped.

bias_mitigation.py

```
def pairwise_compare_debiased(question, response_a, response_b,
                              judge_llm):
    result_1 = judge_llm.invoke(PAIRWISE_PROMPT.format(
        question=question, response_a=response_a,
        response_b=response_b))
    result_2 = judge_llm.invoke(PAIRWISE_PROMPT.format(
        question=question, response_a=response_b,
        response_b=response_a)) # swapped

    grade_1 = parse_grade(result_1)
    grade_2 = parse_grade(result_2)

    if grade_1.winner == "A" and grade_2.winner == "B":
        return "A" # consistent win for A across both orderings
    elif grade_1.winner == "B" and grade_2.winner == "A":
        return "B"
    else:
```

Highest-Leverage Fix

This single technique is the highest-leverage fix in the entire harness: skipping it means your pairwise metric is measuring position bias as much as it measures actual quality.



AI Coach John



FOLLOW

 Repost

Bias Mitigation 2: Verbosity Bias

LLM judges tend to prefer longer answers even when they are not better:

Mitigate by explicitly instructing the judge to penalize unnecessary length and spot-checking against length-matched pairs.

prompts/verbosity.py

```
VERBOSITY_AWARE_PROMPT = PAIRWISE_PROMPT + """
Do NOT favor a response simply because it is longer or more
detailed.
A concise, correct response should be preferred over a longer
response
that includes irrelevant or redundant information.
"""
```

Spot-Check With Synthetic Padding

Beyond prompt instructions, periodically test your judge with a synthetic pair where you have artificially padded one response with irrelevant filler: if your judge consistently prefers the padded version, your prompt needs stronger anti-verbosity language.





Bias Mitigation 3: Self-Preference

A model tends to rate its own outputs more favorably: If you use the same model family to both generate and grade responses, it systematically favors outputs that "sound like itself."

config/models.py

```
# Avoid this pattern:
generator_model = "gpt-5.6"
judge_model = "gpt-5.6"           # same family: risk of self-
preference bias

# Prefer this pattern:
generator_model = "gpt-5.6"
judge_model = "claude-sonnet-5"   # different provider/family for
grading
```



The Adversarial Verifier Pattern

Using a different model (ideally a different provider) as your judge reduces shared blind spots: this is the exact same principle behind the "adversarial verifier" pattern from multi-agent RAG applied to evaluation specifically.



Building the Metric Abstraction

A reusable base class for every metric you will add: Wraps prompt, parsing, and scoring into a clean interface so adding new metrics means writing one small class.

```
metrics/base.py

from abc import ABC, abstractmethod

class Metric(ABC):
    name: str

    @abstractmethod
    def score(self, test_case: TestCase, judge_llm) → dict:
        """Returns {'score': float, 'reasoning': str}"""
        pass

class FaithfulnessMetric(Metric):
    name = "faithfulness"

    def score(self, test_case: TestCase, judge_llm) → dict:
        context = "\n".join(test_case.context or [])
        raw = judge_llm.invoke(
            FAITHFULNESS_PROMPT.format(context=context,
            answer=test_case.actual_output)
        )
        grade = parse_grade(raw)
```



Mirrors how RAGAS and deepeval are structured internally to support dozens of metrics cleanly!



AI Coach John



FOLLOW

 Repost



The Batch **Evaluation Runner**

Running a metric across an entire dataset: Applies one or more metrics across a full list of test cases, collecting results into a structured format ready for aggregation.

runner.py

```
def run_evaluation(test_cases: list[TestCase], metrics:
list[Metric], judge_llm):
    results = []
    for test_case in test_cases:
        row = {"input": test_case.input}
        for metric in metrics:
            result = metric.score(test_case, judge_llm)
            row[f"{metric.name}_score"] = result["score"]
            row[f"{metric.name}_reasoning"] = result["reasoning"]
        results.append(row)
    return results

results = run_evaluation(my_test_cases, [FaithfulnessMetric()],
judge_llm)
```



Start Simple, Then Parallelize

Keep this loop simple and sequential to start: once you are comfortable with the pipeline, parallelizing with **asyncio** or a thread pool is a natural next step to speed up evaluation over large datasets.



Aggregation & Reporting

Turning per-row scores into an actionable summary: Converts raw results into a pandas DataFrame, computes summary stats, and flags worst-performing cases for review.

reporting.py

```
import pandas as pd

def generate_report(results: list[dict]):
    df = pd.DataFrame(results)
    score_columns = [c for c in df.columns if
                     c.endswith("_score")]

    summary = df[score_columns].agg(["mean", "std", "min"])
    worst_cases = df.nsmallest(5, score_columns[0]) # lowest-
    scoring examples

    print(summary)
    return summary, worst_cases

summary, worst = generate_report(results)
```

+ Inspect the Worst Cases, Not Just the Mean

Never report just a single average: the `worst_cases` output is often more useful than the mean, since it directly shows you **which** examples are failing and why (via the stored `reasoning` field), rather than just telling you the aggregate is 0.82.





Calibrating Against Human Judgment

Does your judge actually agree with a real person? Compares the LLM judge's scores against human-labeled examples to measure agreement.

calibration.py

```
from scipy.stats import pearsonr

def calibrate_against_humans(llm_scores: list[float],
                             human_scores: list[float]):
    correlation, p_value = pearsonr(llm_scores, human_scores)
    print(f"Correlation with human judgment: {correlation:.2f} (p={p_value:.3f})")
    return correlation

# Example: label 30 to 50 examples yourself, then compare
correlation = calibrate_against_humans(llm_judge_scores,
                                         human_labels)
```



Benchmark Target: 0.6 to 0.7 Correlation

Do this once whenever you change your judge prompt or judge model: a correlation below ~0.6 to 0.7 against human labels usually means your rubric or prompt needs revision before you can trust the automated scores at scale.



Improving Consistency: Few-Shot Grading

Anchoring the judge with example gradings: Adds labeled examples directly into the prompt showing what a 5, 3, and 1 actually look like.

prompts/few_shot.py

```
FEW_SHOT_FAITHFULNESS_PROMPT = FAITHFULNESS_PROMPT + """
```

```
Examples:
```

```
Context: "Returns accepted within 30 days." Answer: "You can  
return within 30 days."
```

```
Score: 5 (directly supported)
```

```
Context: "Returns accepted within 30 days." Answer: "Returns are  
accepted anytime."
```

```
Score: 1 (directly contradicts context)
```

```
"""
```



Anchoring Tightens Variance

Without anchoring examples, different judge runs can drift in how strictly they interpret the rubric: a handful of well-chosen few-shot examples dramatically tightens this variance, just as few-shot prompting improves generation tasks.



AI Coach John



FOLLOW



Repost

Improving Reliability: Self-Consistency

Reducing noise by asking the judge more than once: Runs the grading prompt multiple times with temperature and averages the results.

voting.py

```
def score_with_self_consistency(test_case, metric, judge_llm,
                                n=3):
    scores = [metric.score(test_case, judge_llm)["score"] for _ in
               range(n)]
    return sum(scores) / len(scores) # simple average; use
    majority vote for categorical

avg_score = score_with_self_consistency(test_case,
    FaithfulnessMetric(), judge_llm, n=3)
```

A Precision Tool, Not Default Setting

This costs more ( extra LLM calls per test case) so reserve it for your smaller, high-stakes calibration and regression sets rather than every single evaluation run: it is a precision tool, not a default setting.



Cost Optimization

Evaluation should not cost more than the system it evaluates: Use smart strategies to scale evaluation without blowing up API budgets.

- ✓ **Use a cheaper judge model:** High-volume, low-stakes checks run great on smaller models; reserve strong judges for regression/calibration.
- ✓ **Sample, do not evaluate everything:** For large production traffic, evaluate a representative random sample rather than 100% of requests.
- ✓ **Cache judge results:** Store results for identical (`test_case` , `metric`) pairs when re-running regression suites without code changes.
- ✓ **Batch API calls:** Group requests where your provider supports batching instead of sequential single calls.

sampler.py

```
import random
sampled_cases = random.sample(all_production_logs, k=200) #
evaluate sample
```



CI/CD Integration: Regression Testing

Running your harness automatically on every prompt change: Wraps evaluation in a script that fails CI pipelines if quality drops below threshold.

```
regression_test.py
```

```
def regression_test(test_cases, metrics, judge_llm,
                    threshold=0.75):
    results = run_evaluation(test_cases, metrics, judge_llm)
    df = pd.DataFrame(results)
    avg_score = df["faithfulness_score"].mean()

    assert avg_score ≥ threshold, f"Faithfulness dropped to
{avg_score:.2f} (threshold: {threshold})"
    print(f"PASSED: faithfulness = {avg_score:.2f}")

# In your CI pipeline (e.g., GitHub Actions):
# python regression_test.py --fail-on-regression
```

An Automated Safety Net

This is what makes an evaluation harness genuinely useful in production: instead of manually re-checking quality after every prompt change, a regression suite catches quality drops automatically, just like unit tests catch code regressions.



The Full Harness, Assembled

All 8 components working together: The complete pipeline from test case definition to CI regression assertion.

main_harness.py

```
# 1. Define test cases
test_cases = [TestCase(input="...", actual_output="...", context=
["..."])]

# 2. Define metrics (pointwise, using bias-aware, few-shot prompts)
metrics = [FaithfulnessMetric()]

# 3. Choose a judge model different from your generator model
judge_llm = get_llm("claude-sonnet-5")

# 4. Run the batch evaluation
results = run_evaluation(test_cases, metrics, judge_llm)

# 5. Aggregate and report
summary, worst_cases = generate_report(results)

# 6. Calibrate against a human-labeled subset periodically
correlation = calibrate_against_humans(llm_scores, human_labels)

# 7. Wrap in a CI regression check
regression_test(test_cases, metrics, judge_llm, threshold=0.75)
```



Structurally identical to RAGAS `evaluate()` or deepeval `assert_test()` !



AI Coach John



FOLLOW

 Repost

When to Use Your Own **vs a Library**

Now that you have built it, when should you actually use it?



Existing Library (RAGAS / deepeval)

- ✓ **Standard metrics:** Fast setup for faithfulness, answer relevancy, and context precision.
- ✓ **Tuned prompts:** Community-maintained prompts tuned across many use cases.
- ✓ **Speed priority:** Setup speed matters more than deep customization.



Custom Harness (Hand-Rolled)

- ✓ **Domain metrics:** Custom checks (e.g., "does this legal answer cite the right statute?").
- ✓ **Full control:** Tight control over bias mitigation, model choice, or compliance audits.
- ✓ **Unique structures:** Evaluating structured queries, agent traces, or tool calls.



Golden Rule: Start with generic libraries to validate your pipeline quickly, then migrate to a custom harness when your custom domain checks and compliance audits require full architectural control.



Common Pitfalls

Mistakes that undermine an otherwise good harness: Watch out for these 7 critical errors.

- ✖ **Skipping position-swap in pairwise checks:** Silently measures position bias, not quality.
- ✖ **Same generator and judge model:** Self-preference bias artificially inflates scores.
- ✖ **Bare numeric score without reasoning:** Chain-of-thought grading is far more reliable.
- ✖ **Never calibrating vs human labels:** No verification if judge agrees with human standards.
- ✖ **Evaluating 100% of production logs:** Unnecessary API costs; sampling is sufficient.
- ✖ **No regression threshold in CI:** Evaluation becomes a one-time check, not a safety net.
- ✖ **Treating average score as whole picture:** Always inspect worst-scoring cases, not just mean.



Production Readiness: Avoid these 7 pitfalls, and your custom evaluation harness will run with high correlation and zero silent regressions.

